

SAP-Assignment 2

Shipping on the Air

Marco Morelli

1. QUALITY ATTRIBUTES

Il sistema descritto in questa relazione è un prototipo di consegna pacchi tramite droni, evoluzione dell' Assignment 01, in cui il design, l'implementazione e il deployment vengono raffinati applicando un sottoinsieme di microservices pattern. Prima di descrivere i pattern vengono individuati due **Quality Attribute (QA)** rappresentativi, di cui si specifica lo scenario: essi costituiscono le due proprietà che i pattern di observability rendono osservabili e verificabili, e sono verificati **end-to-end** secondo quanto descritto nella Sezione 4.

Il primo QA riguarda la **responsiveness** e ne fissa il requisito minimo in condizioni di overload.

QA Scenario: Responsiveness

Quality attribute scenario : Responsiveness

Feature : Small average response time in case of overload
when initiate 1000 concurrent requests
caused by 1000 users
occur in the system
operating in normal operation
then the system processes all requests
so that the average response time is < 100 milliseconds

Il secondo QA riguarda la **gestione dei partial failure**: quando l'account service diventa irraggiungibile, il sistema deve evitare di propagare richieste destinate a fallire.

QA Scenario: Handling partial failures

Quality attribute scenario : Handling partial failures

Feature : Avoid pointless requests in case of unavailability of the account service
when more than 50% of total requests fail
caused by unavailability of the account service
occur in the system
operating in normal operation
then the system makes the requests fail immediately
without propagating to the account service
so that it lightens the workload avoiding pointless requests

I due scenari orientano le scelte architetturali: la responsiveness motiva l'adozione di un modello di I/O non bloccante del gateway e la misura del tempo di risposta a livello di edge come il pattern stesso indica fra le proprie design issue; la gestione dei partial failure motiva il **Circuit Breaker** sull'account proxy del gateway. In entrambi i casi un pattern realizza la proprietà e una metrica **Prometheus** la rende verificabile.

2 DEPLOYMENT

Ogni microservizio è deployato in un singolo container Docker. L'intero sistema è orchestrato con **Docker Compose**, che consente di avviarlo con un solo comando (*docker compose up --build*) a partire dalle immagini buildate direttamente dai rispettivi Dockerfile. Tutti i servizi condividono un'unica rete (*shipping_network*): questa rete condivisa permette di abilitare il **Service Discovery via DNS** e lo scrape di **Prometheus**.

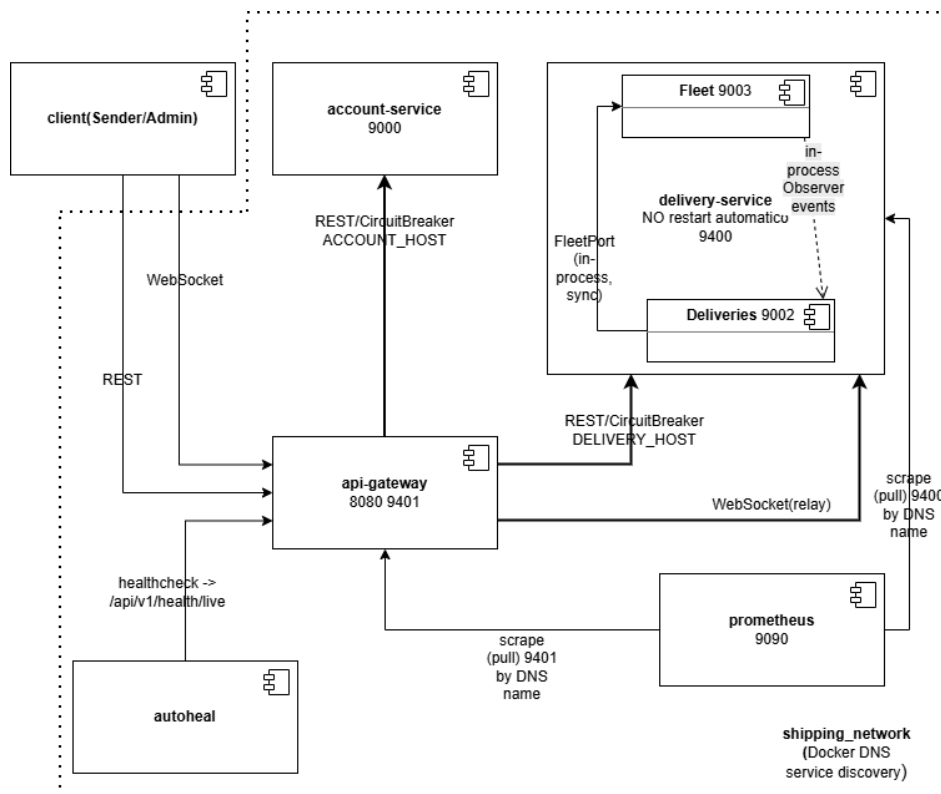
I **container** usano base image *eclipse-temurin-25* e vengono avviati come *fat-jar*. Tale scelta è preferibile all'avvio tramite *mvn exec:java* perché quest'ultimo comporta l'esecuzione di **Maven** a runtime e rende l'avvio del servizio più lento e meno prevedibile. Poiché il container è sottoposto a un *healthcheck* e alla supervisione dell'*autoheal*, un avvio lento può innescare un loop di restart: se l'*healthcheck* inizia a sondare il servizio prima che questo sia pronto a rispondere, il container viene marcato *unhealthy* e riavviato, dando origine a un ciclo in cui il servizio non riesce mai a stabilizzarsi.

La **persistenza** è volontariamente effimera: non viene montato alcun volume, cosicché lo stato scritto a runtime non sopravvive alla ricreazione del container. Sia l'event store del delivery sia il repository degli account ricaricano il proprio file all'avvio, ma in assenza di volume tale file non sopravvive alla ricreazione del container: a ogni *docker compose up --build* lo stato riparte pulito. *AdminSeeder* e *FleetSeeder* riseminano l'account amministratore e i droni, garantendo uno stato deterministico a ogni avvio. È opportuno distinguere i due benefici che la teoria attribuisce all'Event Sourcing: la strutturazione della business logic attorno a eventi con ricostruzione dello stato per replay, e la conservazione durevole della storia dell'aggregato. Il primo è preservato integralmente: l'aggregate Delivery è ricostruito rigiocando i propri eventi, e il replay è reale e verificato dai test. Del secondo, la durabilità oltre il ciclo di vita del container, viene meno non montando alcun volume, coerentemente con la natura di prototipo. La durabilità piena sarebbe ottenibile montando un volume sulla cartella dello store, scelta esclusa per non far sopravvivere una storia di consegne che si disallineerebbe dalla flotta riseminata da zero.

Relativamente alle **restart policy**, è stata adottata una configurazione non uniforme e motivata. I servizi *account-service*, *api-gateway* e *prometheus* adottano *restart: always*, mentre il *delivery-service* non prevede riavvio automatico. La differenza risiede nella natura dello stato dei due servizi applicativi. Lo stato dell'*account-service* è disaccoppiato: un suo riavvio lo riporta a uno stato valido (con l'amministratore riseminato), eventualmente più povero dei sender registrati a runtime, ma senza introdurre incoerenze verso gli altri servizi, di cui resta proprietario delle identità. Per questo il suo riavvio automatico è sicuro. Il *delivery-service* ha invece uno stato accoppiato al runtime solo in parte persistito: le consegne attive sono nell'event store, ma la flotta e i simulatori dei droni sono effimeri e in-memory. In funzionamento normale il servizio non si riavvia mai da solo, e questo scenario non si presenta; è tuttavia proprio la natura di tale stato a motivare la scelta di non abilitare il riavvio automatico. Un riavvio, infatti, ricostruirebbe per replay le sole consegne, mentre la flotta tornerebbe interamente disponibile dal seeder e i simulatori dei droni in volo non ripartirebbero: una consegna immediata resterebbe *IN_PROGRESS* ma congelata, una schedulata non troverebbe più il drone prenotato. Il riavvio automatico presuppone che lo stato del servizio sia interamente recuperabile all'avvio; qui non lo è, perché solo le consegne sono persistite mentre flotta e simulatori sono effimeri. In assenza di questa condizione, un restart non riporterebbe il servizio allo stato precedente al crash, ma lo consoliderebbe in uno stato parziale in cui le consegne referenziano una flotta ripartita da zero. Si preferisce perciò che un eventuale crash resti visibile e richieda un intervento esplicito, anziché essere mascherato da un riavvio che ne fisserebbe l'incoerenza.

3 PATTERNS

Rispetto all'Assignment 1 sono stati applicati i seguenti pattern: **API Gateway, Health Check API, Application Metrics, Event Sourcing, Service Discovery e Circuit Breaker**. L'architettura rimane esagonale con *DDD*: ogni servizio è un progetto Maven autonomo e self-contained. Il sistema finale è composto da tre microservizi deployabili: *account-service*, *delivery-service* e *api-gateway*.



Deployment di sistema

3.1 API Gateway

È stato introdotto un microservizio **api-gateway** come unico endpoint del sistema, con il compito di nascondere la composizione interna dei servizi. Esso espone la **REST API** lato client e, tramite l'*APIGatewayController*, instrada ogni richiesta al servizio a valle sfruttando una routing map e i relativi proxy (*AccountServiceProxy*, *DeliveryServiceProxy*), ereditati e adattati dai proxy del *session-service* dell' Assignment 1.

Il *session-service* di Assignment 1 viene fuso nel gateway. La ragione è che le sue due responsabilità, una volta introdotto il pattern, si collocano entrambe nel gateway. La prima è l'instradamento dei comandi verso i servizi a valle, che il *session-service* svolgeva in assenza di un API layer e che il pattern attribuisce esplicitamente al gateway insieme alla protocol translation. La seconda è la sessione autenticata, sulla quale il servizio applicava authentication e authorization: due edge function che il pattern colloca anch'esse nel gateway, e di cui la sessione è il meccanismo realizzativo.

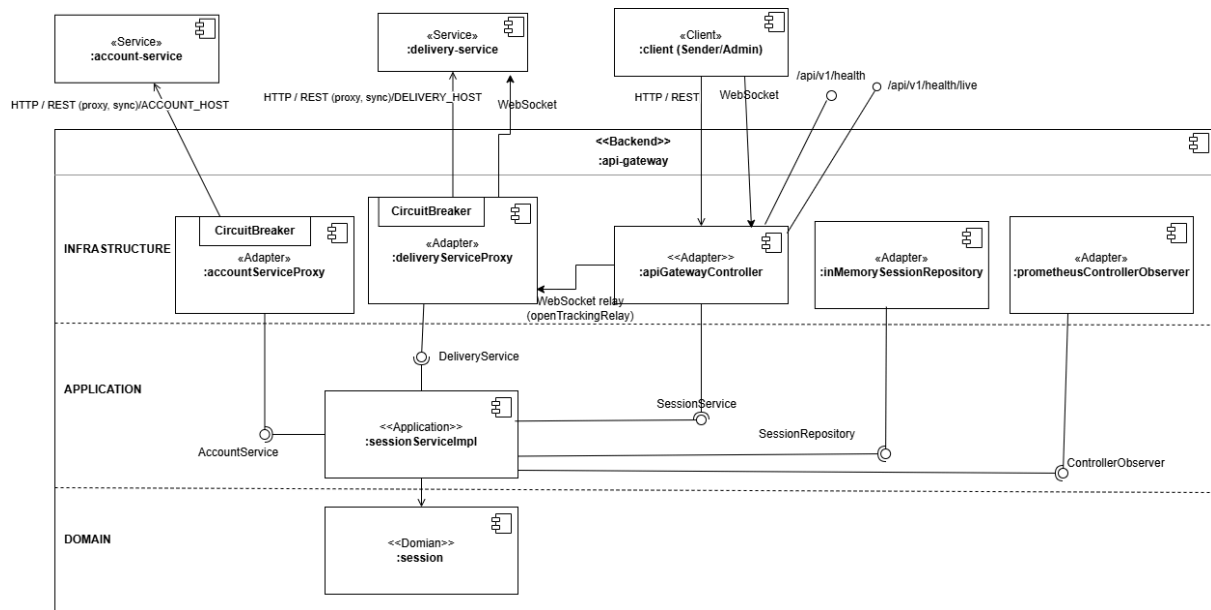
Il vantaggio della fusione è concreto. Mantenere i due componenti separati avrebbe significato un servizio che instrada e autentica davanti a un gateway che instrada e autentica: responsabilità duplicate su due processi,

un hop di rete su ogni comando, e un componente in più da deployare, monitorare e mantenere disponibile. Fondendoli si ottiene un unico endpoint coerente con il pattern, una latenza inferiore e una superficie di rete minore.

La fusione non introduce logica di dominio nell'API layer. Il gateway non compone risposte di più servizi né coordina transazioni distribuite: ogni comando si risolve nell'invocazione di un singolo servizio a valle, secondo una routing map. Il login delega la verifica delle credenziali all'account-service, unico proprietario dei dati di identità, e ne materializza l'esito in una sessione; su ogni comando successivo il gateway risolve l'identificatore di sessione, applica l'autorizzazione per ruolo e ancora al comando l'accountId registrato, anziché fidarsi di un'identità dichiarata dal client nel payload. Le invarianti di consegna e di flotta restano interamente nei servizi che ne sono proprietari.

Il costo, che si documenta, è un gateway "applicativo" che mantiene stato di sessione anziché un puro reverse proxy stateless. Se in futuro emergesse la necessità di API composition o di coordinamento transazionale fra più servizi, essa non andrebbe aggiunta al gateway ma estratta in un mediatore modellato a livello di dominio e collocato a valle di esso.

Il tracking real-time, realizzato precedentemente con una **WebSocket** diretta client-delivery, ora diventa un relay in due tratte: il gateway accetta la WebSocket dal client e apre una WebSocket interna verso il delivery, facendo da relay dei messaggi di posizione ed ETR. Ciò rientra nella protocol translation, che il pattern attribuisce al gateway proprio per la mediazione fra protocolli client-friendly come le WebSocket e i protocolli interni. Il trade-off, che si documenta, è che il gateway mantiene una connessione aperta per ciascun utente che effettua il tracking.



C&C dell'api-gateway

3.2 Observability Patterns

3.2.1 Health Check API

Ogni servizio espone un endpoint di health check. Sul gateway si sono distinti **liveness** e **readiness**:

- *GET /api/v1/health/live (liveness)*: risponde sempre 200/UP senza interrogare i downstream, a indicare che il processo del gateway è attivo.
- *GET /api/v1/health (readiness aggregato)*: sonda account e delivery e risponde 200/UP se entrambi sono disponibili, 503/DOWN altrimenti.

Questa distinzione ha una conseguenza operativa precisa: l'*healthcheck* di **Docker Compose** è configurato sulla *liveness*, cosicché l'*autoheal* riavvia il gateway solo se il processo è effettivamente terminato. Con un unico health aggregato, l'indisponibilità di un downstream avrebbe reso il gateway *unhealthy*, inducendo l'*autoheal* a riavviarlo inutilmente e mascherando così il vero guasto. Il readiness resta invece utile per la diagnosi e per la chiusura del Circuit Breaker.

3.2.2 Application Metrics

Si utilizza **Prometheus**, aggiunto al **Compose** e configurato tramite *prometheus.yml*. Due servizi espongono metriche, ciascuno su una porta dedicata indicata per **DNS name**.

Il Service Discovery abilita lo scrape:

delivery-service

metriche di livello applicativo (porta 9400):

- `created_deliveries`: numero totale di consegne create.
- `deliveries_in_progress`: consegne attualmente in corso.
- `deliveries_delivered`: numero totale di consegne completate.

api-gateway

metriche di livello infrastrutturale (porta 9401):

- `rest_requests`: numero totale di richieste REST.
- `request_response_time_seconds`: tempo di risposta cumulativo in secondi (QA responsiveness).
- `account_circuit_open`: stato del Circuit Breaker dell'account (QA partial failures).

Coerentemente con l'architettura esagonale, la metrica risiede **al di fuori della business logic**: il *delivery-service* impiega l'adapter *PrometheusDeliveryServiceObserver* per esporre le metriche implementando l'interfaccia *DeliveryServiceEventObserver*, mentre il gateway impiega *PrometheusControllerObserver* per le metriche implementando *ControllerObserver*, osservato dall'*APIGatewayController*.

3.3 Event Sourcing

L'Event Sourcing è applicato al **delivery-service**: è il servizio che meglio si presta a essere strutturato a eventi, poiché il ciclo di vita di una consegna è già una sequenza di transizioni di stato per le quali erano stati definiti domain event. L'aggregate *Delivery* è persistito come **sequenza di eventi**, e non come stato: una porta applicativa *EventStore* con adapter *FileBasedEventStore*, e un *EventSourcedDeliveryRepository* che effettua l'append degli eventi e ricostruisce l'aggregate tramite replay.

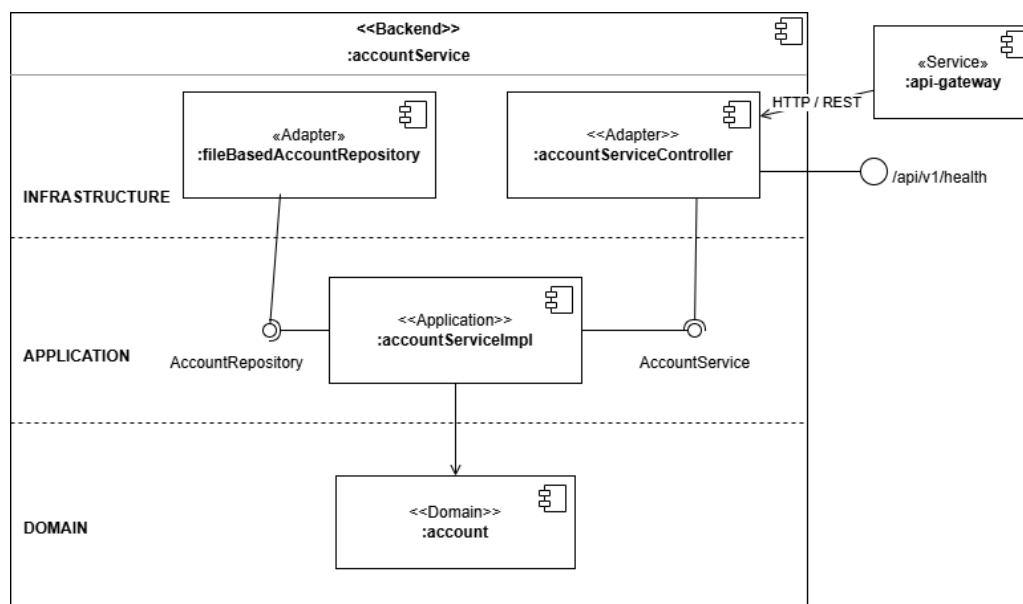
Ogni transizione di stato emette un evento e ogni tipo di evento dispone del proprio ramo in *apply()*, cosicché il replay non può mai fallire. Un solo evento, *EstimatedTimeUpdated*, è escluso dalla persistenza pur essendo emesso sull'aggregate: rappresenta il tempo stimato residuo, che varia a ogni tick del volo e costituisce un dato di **read-model volatile**.

L'*EventSourcedDeliveryRepository* lo filtra esplicitamente in fase di append, cosicché non gonfi lo store senza apportare valore storico. La posizione del drone, *PositionUpdated*, ricade nella stessa categoria di stato volatile ma per una ragione strutturalmente diversa: appartiene al bounded context *fleet*, gestito da un repository in-memory, e non entra affatto nel perimetro dell'event store. Emessa a ogni tick dal Drone, viene consumata in-process

Il **CircuitBreaker** modella tre stati *CLOSED* / *OPEN* / *HALF_OPEN*. Esso traccia i successi e i fallimenti su una finestra con volume minimo: superata la soglia del **50%** di fallimenti, il circuito si apre e il gateway risponde immediatamente con errore senza inoltrare la richiesta. Trascorso un **timeout di 10 secondi**, il breaker passa in *HALF_OPEN* e sonda l'*health* del servizio: se questo è tornato disponibile, il circuito si richiude.

L'aspetto architetturealmente più delicato è **che cosa costituisce un fallimento**. Il breaker conta esclusivamente i guasti del downstream: errori di trasporto o timeout del WebClient e risposte 5xx. Non concorrono invece le risposte applicative 4xx, che indicano che il servizio ha risposto correttamente rifiutando l'input.

Il breaker alimenta la metrica *account_circuit_open* (tramite callback di cambio stato), rendendo osservabile il QA sui partial failure. Il relay WebSocket non attraversa il breaker, trattandosi di uno stream e non di un'interazione request/response sincrona.



C&C dell'account-service

4 TESTING

La strategia di test copre l'intera **test pyramid**. Unit, integration e component test risiedono nel *src/test* di ciascun servizio. Gli end-to-end test risiedono in un modulo separato di soli test (*system-end-to-end*). In ogni servizio sono inoltre presenti test architetturali ArchUnit, che verificano il rispetto dell'architettura esagonale.

4.1 Unit testing

Gli **unit test** seguono la distinzione *solitary/sociable* in base al ruolo della classe. Nel *delivery-service*, ad esempio, il dominio è testato in modalità **sociable** (aggregate Delivery, value object quali peso, coordinate, deadline ed ETR, aggregate Drone), poiché la logica è distribuita tra più oggetti mentre l'application (*DeliveryServiceImpl*) è testata in modalità **solitary**, con le porte mockate.

4.2 Integration testing

Gli **integration test** verificano l'interazione con i sistemi esterni senza avviare l'intera applicazione. Un esempio rappresentativo è il *DeliveryServiceProxy* del gateway verificato contro un delivery service simulato, che ne valida il contratto **REST** verso il servizio a valle. A questo livello si collocano anche i test dell'event store su file, dello scheduler a timer reale, del relay WebSocket, dell'health aggregato e dell'esposizione delle metriche **Prometheus**.

4.3 Component testing

I **component test** verificano un singolo servizio come *black box*, attraverso la sua API, mediante **Cucumber/Gherkin**:

- **account-service**: registrazione, login.
- **delivery-service**: creazione immediata e programmata, rifiuti di dominio, avvio del tracking e viste read-only dell'amministratore su flotta e scheduling.

4.4 End-to-end testing

Gli **end-to-end test** sono richieste esterne verso il sistema completo, effettuate dopo *docker compose up*, secondo l'approccio **user journey**, anch'essi redatti in Gherkin ed eseguiti via Cucumber. Sono definiti tre journey:

- **Creazione delivery**: registrazione → login → creazione immediata → recupero del dettaglio. Esercita la catena gateway → account → delivery.
- **Tracciamento delivery**: (registrazione, login e creazione come precondizioni) → avvio del tracking via relay WebSocket (ricezione di frame reali) → lettura dello status → arresto. Verifica il relay client-gateway-delivery.
- **Cancellazione delivery**: registrazione → login → creazione schedulata → cancellazione delivery.

Nello stesso modulo risiedono i due test che verificano i **QA** scenario della Sezione 1, ed è in questo punto che observability e pattern si compongono in un'unica storia:

- **PerformanceTest (responsiveness)**: invia 1000 richieste concorrenti al gateway e calcola il tempo medio dal *delta* delle metriche, verificando che resti al di sotto di 100 ms. Il probe interroga la liveness, così da misurare la latenza del solo gateway senza falsi 503 sotto carico.
- **CircuitBreakerTest (partial failures)**: disconnette effettivamente l'account dalla rete Docker, osserva la gauge *account_circuit_open* passare a 1, riconnette il servizio e verifica il ritorno a 0. Il gradino 0→1→0 costituisce la prova visiva del pattern.

5 CONCLUSIONI

L'adozione di Docker e Docker Compose ha reso il sistema portabile, riproducibile ed estendibile con un solo comando di avvio; il container per servizio garantisce isolamento e un minore accoppiamento a runtime.

L'applicazione dei pattern ha migliorato il sistema su assi distinti:

- L'API Gateway incapsula la struttura interna dietro un unico endpoint e assorbe la facciata del preesistente session-service.
- Gli observability pattern abilitano il monitoraggio, la diagnosi e l'alerting.
- L'Event Sourcing struttura la persistenza della consegna come storia di eventi.
- Service Discovery e Circuit Breaker accrescono la robustezza del sistema rispetto alla sua natura distribuita.

Il valore architetturale non risiede unicamente nell'aver introdotto i pattern, bensì nell'averli integrati in modo coerente: i due Quality Attribute sono realizzati da un pattern e resi verificabili da una metrica, e la copertura di test lungo l'intera pyramid, culminata negli user journey e nei due test di QA end-to-end, collega la correttezza delle singole funzionalità a quella delle comunicazioni tra i servizi e del sistema nel suo complesso.